

Lab 3: Booting the kernel

Student: Ilya-Linh Nguyen

Innopolis main: i.nguen@innopolis.university

Group: CBS-01

Task

Setup development environment

1. Install git and qemu-system-arm packages
2. Download a mainline linux kernel sources
3. Download a 1st-stage bootloader (any of u-boot, coreboot, etc.) sources
4. Download an embeddable toolset of UNIX programs (busybox, u-root, etc.)
5. Download a GNU cross toolchain

For a vexpress-a9 qemu target

1. Build bootloader for ARM arch (u-boot, coreboot, etc.)
2. Prepare initrd/initramfs (busybox, u-root, etc.)
3. Prepare rootfs (also can be done using busybox, u-root, etc.)
4. Build the mainline linux
5. Run everything in qemu

Solution

Here you can see the workflow of the lab and files (The **out/** dir will be in moodle, it is too large):

- Asciiinema URL: <https://asciiinema.org/a/tvQ3fEUTCdYWpkSN>
- GitHub URL: <https://github.com/ilyalinhnguyen/adv-linux>

Setup development environment

I used a Docker image for this lab because:

- I want to isolate this lab from my system
- It will be easy reproducible in other machines

I put in Docker image all needed packages so let's build it:

```
cd lab3/  
docker build -t adv-linux-lab3 .
```

```

user@laptop:~/Inno/adv-linux$ cd lab3
user@laptop:~/Inno/adv-linux/lab3$ docker build -t adv-linux-lab3 .
[+] Building 2.1s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 584B
=> [internal] load metadata for docker.io/library/ubuntu:24.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/ubuntu:24.04@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
=> resolve docker.io/library/ubuntu:24.04@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
=> CACHED [2/3] RUN apt-get update && apt-get install -y --no-install-recommends bash ca-certificates curl xz-utils bzip2 gzip tar unzip file cpio git make gcc g++ rsync
=> CACHED [3/3] WORKDIR /work
=> exporting to image
=> => exporting layers
=> exporting manifest sha256:b644da4aad029b2320d17054eb37dbcb34e8ca810a095b531d2ea0fb3ef341f3
=> exporting config sha256:b87da342d7c7b9e5b80206b68745fe72b01339cb74a9c8d07233cadea14b2aad
=> exporting attestation manifest sha256:fddeecdde7f9d5d17faaa4ae45685d1cf330a31a1fa1d3e871425593833a8b4
=> exporting manifest list sha256:b387d67e4ec45353b8bb52ef4fc4282309f12f8afddb1c38a522a0470ef1c61f
=> naming to docker.io/library/adv-linux-lab3:latest
=> unpacking to docker.io/library/adv-linux-lab3:latest

```

And run it

```
docker run --rm -it --privileged -v "/home/user/Inno/adv-
linux/lab3:/work/lab3" adv-linux-lab3 bash
```

We are inside our container.

Let's prepare folders and environment variables:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export MAKEFLAGS="-j$(nproc)"

mkdir -p /work/lab3/src /work/lab3/out
```

Perfect, now let's clone the sources:

```
cd src/
git clone --depth 1 --branch v2022.01 https://github.com/u-boot/u-boot.git
git clone --depth 1 --branch v6.6 https://github.com/torvalds/linux.git
git clone --depth 1 --branch 1_36_1 https://github.com/mirror/busybox.git
```

After a long time everything is cloned.

Now let's build U-boot:

```
cd u-boot/
make vexpress_ca9x4_defconfig
make
cp -f u-boot /work/lab3/out/u-boot
```

```

root@1418fafa0f49: /work/lab3/src/u-boot
CC      lib/efi_loader/efi_variable.o
CC      lib/crc32.o
CC      lib/efi_loader/efi_watchdog.o
CC      lib/efi_loader/efi_disk.o
CC      lib/ctype.o
CC      lib/div64.o
CC      lib/efi_loader/efi_net.o
CC      lib/hang.o
CC      lib/linux_compat.o
CC      lib/efi_loader/efi_smbios.o
CC      lib/linux_string.o
CC      lib/efi_loader/efi_load_initrd.o
CC      lib/lmb.o
CC      lib/membuff.o
CC      lib/efi_loader/helloworld.o
AS      lib/efi_loader/efi_crt0.o
CC      lib/slr.o
CC      lib/string.o
CC      lib/efi_loader/efi_reloc.o
CC      lib/efi_loader/efi_freestanding.o
CC      lib/tables_csum.o
CC      lib/tlme.o
CC      lib/hexdump.o
CC      lib/uuid.o
CC      lib/rand.o
CC      lib/panic.o
CC      lib/vsprintf.o
LD      lib/efi_loader/helloworld_efi.so
arm-linux-gnueabi-lld.bfd: warning: lib/efi_loader/efi_crt0.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabi-lld.bfd: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
arm-linux-gnueabi-lld.bfd: warning: lib/efi_loader/helloworld_efi.so has a LOAD segment with RWX permissions
CC      lib/strto.o
CC      lib/abuf.o
OBJCOPY lib/efi_loader/helloworld.efi
CC      lib/date.o
CC      lib/rtc-lib.o
CC      lib/elf.o
AR      lib/efi_loader/built-in.o
AR      fs/ubifs/built-in.o
AR      fs/built-in.o
AR      lib/built-in.o
LD      u-boot
arm-linux-gnueabi-lld.bfd: warning: arch/arm/lib/libifuncs.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabi-lld.bfd: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
arm-linux-gnueabi-lld.bfd: warning: u-boot has a LOAD segment with RWX permissions
OBJCOPY u-boot.srec
OBJCOPY u-boot.nodtb.bin
SYM     u-boot.sym
COPY    u-boot.bin
root@1418fafa0f49: /work/lab3/src/u-boot#

```

Build Linux kernel:

```

cd /work/lab3/src/linux
make vexpress_defconfig
./scripts/config --enable DEVTMPFS
./scripts/config --enable DEVTMPFS_MOUNT
./scripts/config --enable BLK_DEV_INITRD
./scripts/config --enable EXT4_FS
./scripts/config --enable VIRTIO
./scripts/config --enable VIRTIO_MMIO
./scripts/config --enable VIRTIO_BLK
make olddefconfig
make zImage dtbs
cp -f arch/arm/boot/zImage /work/lab3/out/zImage
cp -f arch/arm/boot/dts/arm/vexpress-v2p-ca9.dtb /work/lab3/out/vexpress-
v2p-ca9.dtb

```

```
root@1418fafa0f49: /work/lab3/src/linux
CC      fs/mbcache.o
CC      fs/coredump.o
CC      fs/drop_caches.o
CC      fs/sysctl.o
CC      fs/fhandle.o
AR      drivers/hid/built-in.a
AR      drivers/built-in.a
AR      fs/built-in.a
AR      built-in.a
AR      vmlinux.a
LD      vmlinux.o
OBJCOPY modules.builtin.modinfo
GEN      modules.builtin
MODPOST vmlinux.symvers
CC      .vmlinux.export.o
UPD      include/generated/utsversion.h
CC      init/version-timestamp.o
LD      .tmp_vmlinux.kallsyms1
NM      .tmp_vmlinux.kallsyms1.syms
KSYM    .tmp_vmlinux.kallsyms1.5
AS      .tmp_vmlinux.kallsyms1.5
LD      .tmp_vmlinux.kallsyms2
NM      .tmp_vmlinux.kallsyms2.syms
KSYM    .tmp_vmlinux.kallsyms2.5
AS      .tmp_vmlinux.kallsyms2.5
LD      vmlinux
NM      System.map
SORTTAB vmlinux
OBJCOPY arch/arm/boot/Image
Kernel: arch/arm/boot/Image is ready
LDS      arch/arm/boot/compressed/vmlinux.lds
AS      arch/arm/boot/compressed/head.o
GZIP     arch/arm/boot/compressed/piggy_data
CC      arch/arm/boot/compressed/misc.o
CC      arch/arm/boot/compressed/decompress.o
CC      arch/arm/boot/compressed/string.o
AS      arch/arm/boot/compressed/hyp-stub.o
CC      arch/arm/boot/compressed/fdt_rw.o
CC      arch/arm/boot/compressed/fdt_ro.o
CC      arch/arm/boot/compressed/fdt_wip.o
CC      arch/arm/boot/compressed/fdt.o
CC      arch/arm/boot/compressed/fdt_check_mem_start.o
AS      arch/arm/boot/compressed/lib1funcs.o
AS      arch/arm/boot/compressed/ashldi3.o
AS      arch/arm/boot/compressed/bswapsdi2.o
AS      arch/arm/boot/compressed/piggy.o
LD      arch/arm/boot/compressed/vmlinux
OBJCOPY arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready
oot@1418fafa0f49: /work/lab3/src/linux#
```

And build busybox (I used `sed` for more comfortable editing):

```
cd /work/lab3/src/busybox
make distclean
make defconfig
sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
sed -i 's/# CONFIG_FEATURE_SH_STANDALONE is not
set/CONFIG_FEATURE_SH_STANDALONE=y/' .config
sed -i 's/# CONFIG_FEATURE_SH_NOFORK is not
set/CONFIG_FEATURE_SH_NOFORK=y/' .config
sed -i 's/^CONFIG_TC=.*#/CONFIG_TC is not set/' .config
make oldconfig </dev/null
make
```

```

root@1418fafa0f49: /work/lab3/src/busybox
CC libbb/securetty.o
CC libbb/setup_environment.o
CC libbb/signals.o
CC libbb/simplify_path.o
CC libbb/single_argv.o
CC libbb/skip_whitespace.o
CC libbb/speed_table.o
CC libbb/str_tolower.o
CC libbb/strrstr.o
CC libbb/sysconf.o
CC libbb/time.o
CC libbb/trim.o
CC libbb/u_signal_names.o
CC libbb/ubi.o
CC libbb/udp_io.o
CC libbb/unicode.o
CC libbb/update_passwd.o
CC libbb/utmp.o
CC libbb/uencode.o
CC libbb/verror_msg.o
CC libbb/vfork_daemon_rexec.o
CC libbb/warn_ignoring_args.o
CC libbb/wfopen.o
CC libbb/wfopen_input.o
CC libbb/write.o
CC libbb/xatnum.o
libbb/update_passwd.c: In function 'update_passwd':
libbb/update_passwd.c:153:17: warning: ignoring return value of 'fchown' declared with attribute 'warn_unused_result' [-Wunused-result]
 153 |         fchown(new_fd, sb.st_uid, sb.st_gid);
      |         ^~~~~~
CC libbb/xconnect.o
CC libbb/xfunc_die.o
CC libbb/xfuncs.o
CC libbb/xfuncs_printf.o
CC libbb/xgetcwd.o
CC libbb/xgethostbyname.o
CC libbb/xreadlink.o
CC libbb/xrealloc_vector.o
CC libbb/xregcomp.o
AR libbb/lib.a
LINK busybox_unstripped
static linking against glibc, can't use --gc-sections
trying libraries: m resolv rt
Library m is needed, can't exclude it (yet)
Library resolv is needed, can't exclude it (yet)
Library rt is not needed, excluding it
Library m is needed, can't exclude it (yet)
Library resolv is needed, can't exclude it (yet)
final link with: m resolv
root@1418fafa0f49: /work/lab3/src/busybox#

```

Now let's create initramfs.

```
make CONFIG_PREFIX=/work/lab3/out/initramfs install
mkdir -p /work/lab3/out/initramfs/{proc,sys,dev,newroot}
```

And write the following `init` script:

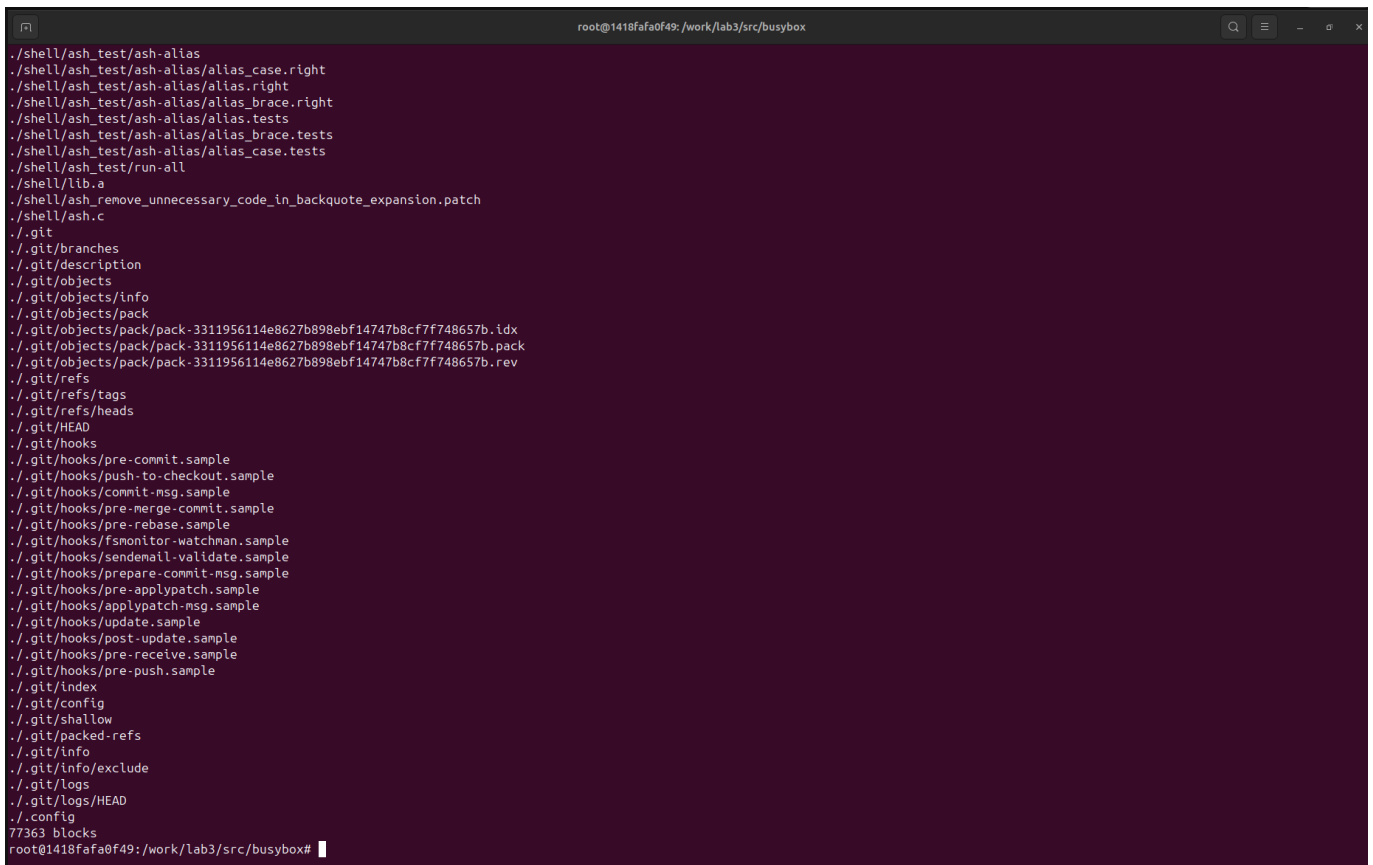
```
#!/bin/sh
set -e
mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev
echo "[initramfs] early userspace started"
echo "[initramfs] waiting for /dev/mmcblk0..."
for _ in 1 2 3 4 5 6 7 8 9 10; do
    [ -b /dev/mmcblk0 ] && break
    sleep 1
done
mount -t ext4 /dev/mmcblk0 /newroot
echo "[initramfs] switched to rootfs on /dev/mmcblk0"
exec switch_root /newroot /sbin/init
```

Giving rights for run it:

```
chmod +x /work/lab3/out/initramfs/init
```

And package our `initramfs/` directory into a compressed initramfs image so the Linux kernel can unpack it at boot:

```
find . -print0 | cpio --null -ov --format=newc >
/work/lab3/out/initramfs.cpio
gzip -f /work/lab3/out/initramfs.cpio
```



```
root@1418fafa0f49: /work/lab3/src/busybox
./shell/ash_test/ash-alias
./shell/ash_test/ash-alias/alias_case.right
./shell/ash_test/ash-alias/alias.right
./shell/ash_test/ash-alias/alias_brace.right
./shell/ash_test/ash-alias/alias_tests
./shell/ash_test/ash-alias/alias_brace_tests
./shell/ash_test/ash-alias/alias_case_tests
./shell/ash_test/run-all
./shell/lib.a
./shell/ash_remove_unnecessary_code_in_backquote_expansion.patch
./shell/ash.c
./git
./git/branches
./git/description
./git/objects
./git/objects/info
./git/objects/pack
./git/objects/pack/pack-3311956114e8627b898ebf14747b8cf7f748657b.idx
./git/objects/pack/pack-3311956114e8627b898ebf14747b8cf7f748657b.pack
./git/objects/pack/pack-3311956114e8627b898ebf14747b8cf7f748657b.rev
./git/refs
./git/refs/tags
./git/refs/heads
./git/HEAD
./git/hooks
./git/hooks/pre-commit.sample
./git/hooks/push-to-checkout.sample
./git/hooks/commit-msg.sample
./git/hooks/pre-merge-commit.sample
./git/hooks/pre-rebase.sample
./git/hooks/fsmonitor-watchman.sample
./git/hooks/sendemail-validate.sample
./git/hooks/prepare-commit-msg.sample
./git/hooks/pre-applypatch.sample
./git/hooks/applypatch-msg.sample
./git/hooks/update.sample
./git/hooks/post-update.sample
./git/hooks/pre-receive.sample
./git/hooks/pre-push.sample
./git/index
./git/config
./git/shallow
./git/packed-refs
./git/info
./git/info/exclude
./git/logs
./git/logs/HEAD
./config
77363 blocks
root@1418fafa0f49: /work/lab3/src/busybox#
```

Now let's create rootfs and ext4 image:

```
mkdir -p /work/lab3/out/rootfs
make CONFIG_PREFIX=/work/lab3/out/rootfs install
mkdir -p /work/lab3/out/rootfs/{proc,sys,dev,etc,root,tmp,mnt}
```

And write the following `init` script:

```
#!/bin/sh
mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev
echo "[rootfs] root filesystem is mounted"
```

```
echo "[rootfs] boot flow: U-Boot -> Linux kernel -> initramfs -> rootfs"
exec /bin/sh
```

Giving rights to it:

```
chmod +x /work/lab3/out/rootfs/sbin/init
```

Create a file that will be virtual SD card/disk with 128 MB:

```
truncate -s 128M /work/lab3/out/rootfs.ext4
```

Formats that file with an ext4 filesystem:

```
mkfs.ext4 -F /work/lab3/out/rootfs.ext4
```

Create mountpoint dir and mount the ext4 filesystem using a loop device, so the kernel treats the file like a disk partition:

```
mkdir -p /tmp/rootfs-mnt
mount -o loop /work/lab3/out/rootfs.ext4 /tmp/rootfs-mnt
```

Copy my prepared rootfs directory tree into the mounted filesystem and unmount the image cleanly and remove the temporary mount directory:

```
cp -a /work/lab3/out/rootfs/. /tmp/rootfs-mnt/
umount /tmp/rootfs-mnt
rmdir /tmp/rootfs-mnt
```

So let's boot in QEMU now:

```
INITRD_SIZE_HEX=$(printf "0x%x" "${stat -c %s
/work/lab3/out/initramfs.cpio.gz}")
( sleep 3; \
  echo "setenv bootargs console=ttyAMA0 root=/dev/mmcblk0 rw rdinit=/init"; \
  \
  echo "bootz 0x60010000 0x63000000:${INITRD_SIZE_HEX} 0x62f00000"; \
) | timeout 120 qemu-system-arm \
  -M vexpress-a9 \
  -m 512M \
  -nographic \
```

```

-kernel /work/lab3/out/u-boot \
-device loader,file=/work/lab3/out/zImage,addr=0x60010000 \
-device loader,file=/work/lab3/out/initramfs.cpio.gz,addr=0x63000000
\
-device loader,file=/work/lab3/out/vexpress-v2p-
ca9.dtb,addr=0x62f00000 \
-drive file=/work/lab3/out/rootfs.ext4,if=sd,format=raw

```

QEMU starts by executing the U-Boot binary as if it were the firmware/bootloader for the **vexpress-a9** machine. I use **-nographic** so all output goes to the terminal via the emulated serial (**ttYAMA0**).

-device loader pre-loads **zImage**, **initramfs.cpio.gz**, and the **dtb** into RAM at the given addresses; then we "type" U-Boot commands: **setenv bootargs ... rdinit=/init** and **bootz KERNEL INITRD:SIZE DTB**. Rootfs is attached as **if=sd**, so Linux sees it as **/dev/mmcblk0** (mounted by **initramfs**, then **switch_root**). **timeout 120** just stops QEMU after 2 minutes.

```

root@2453912b4c5a: /work/lab3/src/busybox
9pnet: Installing 9P2000 support
Registering SWP/SWPB emulation handler
10009000.serial: ttYAMA0 at MMIO 0x10009000 (irq = 38, base_baud = 0) is a PL011 rev1
printk: console [ttYAMA0] enabled
1000a000.serial: ttYAMA1 at MMIO 0x1000a000 (irq = 39, base_baud = 0) is a PL011 rev1
1000b000.serial: ttYAMA2 at MMIO 0x1000b000 (irq = 40, base_baud = 0) is a PL011 rev1
1000c000.serial: ttYAMA3 at MMIO 0x1000c000 (irq = 41, base_baud = 0) is a PL011 rev1
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:iofpga7,00000000/10007000.kni/seri0/input/i
nput0
mmc0: new SD card at address 4567
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
mmcblk0: mmc0:4567 QEMU! 128 MiB
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
clk: Disabling unused clocks
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 37
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process
[initramfs] early userspace started
[initramfs] waiting for /dev/mmcblk0...
EXT4-fs (mmcblk0): mounted filesystem 686b0281-a67b-41cf-ba97-fd5714187297 r/w with ordered data mode. Quota mode: disabled.
[initramfs] switched to rootfs on /dev/mmcblk0
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:iofpga7,00000000/10007000.kni/seri
o1/input/input2
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
random: crng init done
[rootfs] root filesystem is mounted
[rootfs] boot flow: U-Boot -> Linux kernel -> initramfs -> rootfs
/bin/sh: can't access tty; job control turned off
~ # drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
amba 100e0000.memory-controller: deferred probe pending
amba 100e1000.memory-controller: deferred probe pending
amba 100e5000.watchdog: deferred probe pending
i2c 0-0039: deferred probe pending
amba 1000f000.wdt: deferred probe pending
amba 10020000.clcd: deferred probe pending

```

That's it for the task, **out/** directory can be accessed here:

https://drive.google.com/drive/folders/1Z_OLO3qbcD9kafouvRshc6e7v6DiKYcN?usp=sharing